

Exercícios Capacitação Python - Dia 3

Aquecimento

1. Crie uma lista vazia usando a sintaxe com colchetes (ex: `a = []`). Utilizando um laço `for` em conjunto com a função `range(5)` para gerar um intervalo, repita um bloco de código. Dentro do laço, peça para o usuário digitar um número inteiro com `input()` e use o método `append(item)` para adicionar este número ao final da lista. Ao final, imprima a lista completa.
2. A função `range()` pode receber argumentos de início, fim e passo (`range(inicio, fim, passo)`). Construa um laço `for` usando o intervalo adequado para iterar sobre todos os números pares entre 0 e 20 (garantindo que o passo seja 2). Para cada repetição, utilize a função `print()` para exibir o número na tela.
3. Inicialize uma variável booleana chamada `continuar` com o valor `True`. Crie um laço `while` usando essa variável booleana diretamente na condição. Dentro do laço, peça ao usuário para digitar uma palavra através do `input()`. Se a palavra digitada for exatamente "sair", altere o valor da variável `continuar` para `False` dentro de um bloco `if` para interromper as repetições.
4. Inicialize a lista `[10, 20, 30, 40, 50]`. Através da indexação (usando `nome_da_lista[i]`), acesse o elemento que se encontra no índice 2 e substitua/modifique o seu valor para 100. Em seguida, utilize o método `pop()` para remover o último elemento da lista, armazenando o valor retornado em uma variável. Imprima a lista modificada e o valor que foi removido.
5. Dada a lista de strings `["Abacaxi", "Uva", "Romã", "Abacate"]`, utilize o laço no formato `for fruta in lista:` para percorrer cada um dos elementos. Dentro do laço de repetição, verifique com um bloco `if` e o comparador de igualdade (`==`) se o elemento da iteração atual é "Uva". Se for, imprima "Achei a Uva!", caso contrário, imprima apenas o nome da fruta percorrida.

Avançando

6. Inicialize uma lista preenchida com a repetição de um valor zero, usando o formato de multiplicação de listas (ex: `[0] * 5`). Use a função `len(lista)` combinada com a função `range()` em um laço `for` para percorrer todos os índices da lista gerada. A cada repetição, acesse a posição atual e altere o valor nela armazenado para ser igual ao dobro do valor numérico do seu próprio índice (usando o operador `*` de matemática). Imprima o resultado final da lista.
7. Crie duas listas vazias: uma para números pares e outra para números ímpares. Usando um laço que execute repetições de forma indefinida, como `while True:`, receba números inteiros do usuário. Verifique se o número inserido é 0; caso seja, modifique a condição ou encerre o laço. Se não for 0, use o operador de módulo (`%`)

abordado nas aulas anteriores para analisar o resto da divisão por 2. Dependendo da condicional ser par ou ímpar, use `append()` para guardar o número na respectiva lista e as imprima no final.

8. Inicialize a lista `[5, 2, 9, 1, 5, 6]`. Aplique o método `remove(5)` para localizar e remover a primeira ocorrência do número cinco na estrutura. Após a remoção, utilize o método `insert(i, item)` especificando o índice 2 para inserir o número 10 no meio da lista, empurrando os subsequentes para frente. Por fim, use o método `.sort()` para ordenar todos os elementos em ordem crescente e imprima a lista pronta.

9. Peça ao usuário que insira seu nome completo usando a função de entrada de dados e salve isso numa variável. Tendo em vista que podemos acessar "pedaços" de cadeias de caracteres usando o fatiamento (`[n:m]`) de forma similar às listas, utilize essa operação para extrair do índice 0 até a posição 4 (não inclusa) e imprima a string resultante. Além disso, use a função nativa `len()` para contar e exibir a quantidade total de caracteres digitados.

10. Crie a lista `[1, 2, 3]` e também a lista `[4, 5, 6]`, armazenando-as em variáveis separadas. Realize a operação de concatenação unindo as listas através do operador de adição (+), salvando o resultado em uma nova estrutura. Após a concatenação, aplique o método específico `reverse()` para modificar e inverter toda a ordem dos elementos numéricos presentes. Imprima a lista final na tela.

Desafios

11. Receba e armazene um valor inteiro inserido pelo usuário. Crie uma lista em branco. Utilizando um laço de repetição `while` cuja condição verifique se o número é maior que 0, extraia os dígitos individualmente de trás para frente usando o resto da divisão (`% 10`). Adicione esse dígito encontrado na lista por meio do método `append()`. Antes do laço se repetir, atualize o valor do número realizando uma divisão inteira (`// 10`). Imprima a lista contendo os números fatiados.

12. Crie uma lista com 6 números, preenchendo-a sequencialmente ao ler os dados do usuário dentro de um laço `for`. É estritamente proibido utilizar o método pronto `.reverse()` nesta questão. Construa a lógica através de um segundo laço de repetição acessando os índices negativos sequencialmente (`-1, -2, ...`) e inserindo esses valores por `append()` em uma segunda lista, imprimindo-a na ordem reversa à fornecida.

13. O conceito da sequência de Fibonacci diz que os próximos números sempre consistem na soma dos dois valores que os antecedem. Inicie seu código declarando explicitamente a lista `[0, 1]`. Peça que o usuário insira um número limite L. Use um laço condicional `while` exigindo que a instrução de tamanho `len(lista)` permaneça menor que L. A cada volta, some os elementos armazenados nos dois últimos índices e guarde o resultado da soma usando `append()`, até completar a estrutura.

14. Retomando o código da aula que verifica se um número individual é primo. Amplie aquela mecânica: exija do usuário um valor positivo X. Empregue dois laços de repetição, onde o externo percorra do número 2 até o valor X através do intervalo de `range()`, e o laço interno faça o cálculo com módulo (%) e comparador de igualdade (==) procurando por divisores exatos. Guarde os números verdadeiramente primos em uma lista através do `append()` e os imprima.